

C CONSTANTS

IMMUTABLE DATA & SYMBOLIC LOGIC

A 12-Page Comprehensive Guide

01. Introduction to Constants

In C programming, a **Constant** is a value that remains fixed throughout the lifetime of the program. Unlike variables, which act as dynamic containers, constants act as hardcoded identities.

Definition: A constant is an identifier whose value cannot be altered during the execution of a program.

Constants are used to enhance security, prevent accidental data modification, and make code more readable by using descriptive names for fixed values.

02. Integer Constants

These are whole numbers without any fractional or decimal parts. They can be positive, negative, or zero.

```
int x = 100; // 100 is the constant
```

Technical Variations:

- **Decimal:** Base 10 (e.g., 10, -50).
- **Octal:** Base 8, prefixed with 0 (e.g., 012).
- **Hexadecimal:** Base 16, prefixed with 0x (e.g., 0x1A).

03. Floating-Point Constants

Also known as Real Constants, these represent numbers that contain decimal points or fractional parts.

```
float pi = 3.14159;
```

Representation Modes:

- **Fractional:** Standard decimal form (e.g., 2.50).
- **Exponential:** Scientific notation using 'e' or 'E' (e.g., 2e3 which means 2000).

04. Character Constants

A character constant is a single alphabet, digit, or special symbol enclosed within **single quotes** (' ').

```
char grade = 'A';
```

Under the Hood: C stores these as integers based on their **ASCII** (American Standard Code for Information Interchange) values. For example, 'A' is stored as 65.

05. String Constants

A string constant is a sequence of characters enclosed within **double quotes** ("").

```
printf("Hello World"); // String Constant
```

The Null Terminator: C automatically adds a hidden `\0` (null character) at the end of every string constant to signal the end of the text in memory.

06. The const Keyword

The `const` keyword is a qualifier that tells the compiler that the variable's value is non-modifiable after initialization.

```
const int MAX_USERS = 50;
```

If you try to write `MAX_USERS = 60;` later in the code, the compiler will throw an error immediately, ensuring data integrity.

07. Symbolic Constants

Constants can also be created using the `#define` preprocessor directive. These are defined at the very top of your code.

```
#define GOLDEN_RATIO 1.618
```

Mechanism: Before the code is compiled, the preprocessor finds every instance of `GOLDEN_RATIO` and replaces it with `1.618`. It does not occupy variable memory.

08. Variable vs Constant

Feature	Variable	Constant
Mutability	Values can change	Values are frozen
Syntax	<code>int x = 5;</code>	<code>const int x = 5;</code>
Memory	Stored in RAM	May be hardcoded

09. Enum Constants (Introduction)

The enum keyword allows you to create a set of named integer constants, making the code much more readable.

```
enum Days {MON, TUE, WED};
```

Here, MON is 0, TUE is 1, and so on. This is a cleaner way to handle categorized constants.

10. Pitfalls & Best Practices

- **Initialization:** You must initialize a `const` variable at the time of declaration.
- **Naming:** It is standard industry practice to name constants in **ALL_CAPS**.
- **Precision:** Using `#define` doesn't check for data types, while `const` does.

11. Final Summary

- **Literal Constants:** Hardcoded values like 5 or 'Z'.
- **Qualified Constants:** Using the const keyword.
- **Symbolic Constants:** Using #define.
- Constants ensure code remains stable and easy to maintain.

END OF C CONSTANTS GUIDE